

# Bug serializers : `validated_data` utilise les mauvaises clés

Date : 2026-05-14 Severité : **Critique** — cause des `KeyError` 500 sur tous les POST/PUT/PATCH Dernière vérification : 2026-05-14 après merge de `origin/production` dans `main` Concerne : **13 serializers** dans le backend `holly_pi` Non contournable côté frontend — le bug est dans le code Python du serializer

## Le problème en détail

### Contexte : comment fonctionne `source` dans DRF

Django REST Framework permet de renommer les champs entre l'API (JSON) et le modèle Django grâce au paramètre `source`. Par exemple :

```
class ArticleSerializer(serializers.ModelSerializer):
    name = serializers.CharField(source='nom')          # API: "name" → modèle: "nom"
    price = serializers.DecimalField(source='prix')     # API: "price" → modèle: "prix"
```

Cela signifie :

- **En lecture (GET)** : DRF lit `instance.nom` et le renvoie sous la clé `"name"` dans le JSON
- **En écriture (POST/PUT)** : DRF reçoit `"name"` dans le JSON, valide la valeur, puis la stocke dans `validated_data` sous la clé du `source` ( `'nom'` ), pas sous la clé API ( `'name'` )

### Le bug

Les méthodes `create()` et `update()` custom accèdent à `validated_data` avec les **noms API** au lieu des **noms source** (**modèle**). Or DRF a déjà fait la transformation.

### Démonstration concrète

```
# Déclaration du serializer
class ArticleSerializer(serializers.ModelSerializer):
    name = serializers.CharField(source='nom')

# Ce que le frontend envoie :
POST /api/articles/ {"name": "Recette A", "price": "10", "categorie_id": 1}

# Ce que DRF fait en interne après validation :
# 1. Il reçoit {"name": "Recette A"}
# 2. Il valide la valeur "Recette A" pour le champ `name`
# 3. Il stocke le résultat dans validated_data sous la clé SOURCE :
#     validated_data = {'nom': 'Recette A', 'prix': Decimal('10'), 'categorie': <CategorieArticle>}
#                       ^^^               ^^^
#                       clé = source      clé = source
#                       PAS 'name'        PAS 'price'

# Le bug dans create() :
def create(self, validated_data):
    return Article.objects.create(
        nom=validated_data['name'],    # ← KeyError ! 'name' n'existe pas dans validated_data
        prix=validated_data['price'],  # ← KeyError ! 'price' n'existe pas non plus
    )

# La version correcte :
def create(self, validated_data):
```

```

return Article.objects.create(
    nom=validated_data['nom'],      # ← OK
    prix=validated_data['prix'],    # ← OK
)

```

## Preuve par test unitaire

Nous avons exécuté ce test directement sur le backend `holly_pi` :

```

>>> from apps.menu.serializers import ArticleDetailSerializer
>>> s = ArticleDetailSerializer(data={
...     'name': 'Test', 'price': '10', 'categorie_id': 1, 'description': 'test'
... })
>>> s.is_valid()
True
>>> s.validated_data
{'nom': 'Test', 'categorie': <CategorieArticle: Entrées>, 'prix': Decimal('10.00'), 'description':
'test'}

```

→ `validated_data` contient bien `'nom'` et `'prix'`, jamais `'name'` ni `'price'`.

## Pourquoi ce n'est pas contournable côté frontend

Le frontend envoie les bons noms de champs ( `name` , `price` , `categorie_id` ). Le serializer les accepte correctement. La transformation `source` est faite à l'intérieur de DRF, entre la validation et l'appel à `create()` . Le frontend n'a aucun contrôle sur les clés de `validated_data` .

## Statut après merge production (2026-05-14)

### Corrigés

| Serializer                   | Fichier                       | Méthode de correction                            |
|------------------------------|-------------------------------|--------------------------------------------------|
| <b>ShiftSerializer</b>       | apps/planning/serializers.py  | create()/update() custom supprimés → DRF default |
| <b>FournisseurSerializer</b> | apps/suppliers/serializers.py | create()/update() custom supprimés → DRF default |

### Toujours cassés (13 serializers)

| Serializer                  | Fichier                          |
|-----------------------------|----------------------------------|
| CategorieArticleSerializer  | apps/menu/serializers.py         |
| ArticleSerializer           | apps/menu/serializers.py         |
| ArticleDetailSerializer     | apps/menu/serializers.py         |
| ArticleIngredientSerializer | apps/menu/serializers.py         |
| IngredientSerializer        | apps/inventory/serializers.py    |
| StockSerializer             | apps/inventory/serializers.py    |
| RestaurantSerializer        | apps/restaurant/serializers.py   |
| SalleSerializer             | apps/salles/serializers.py       |
| ReservationSerializer       | apps/reservations/serializers.py |

|                               |                               |
|-------------------------------|-------------------------------|
| CommandeFournisseurSerializer | apps/suppliers/serializers.py |
| FactureSerializer             | apps/billing/serializers.py   |
| PaieementSerializer           | apps/billing/serializers.py   |
| ReportSerializer              | apps/reports/serializers.py   |

## Détail par fichier

### 1. apps/menu/serializers.py

#### CategorieArticleSerializer

| Méthode    | Clé utilisée (faux)                 | Clé correcte (source)                 |
|------------|-------------------------------------|---------------------------------------|
| validate() | data.get('name')                    | data.get('nom')                       |
| create()   | validated_data['name']              | validated_data['nom']                 |
| create()   | validated_data.get('display_order') | validated_data.get('ordre_affichage') |
| update()   | validated_data.get('name')          | validated_data.get('nom')             |
| update()   | validated_data.get('display_order') | validated_data.get('ordre_affichage') |

#### ArticleSerializer

| Méthode  | Clé utilisée (faux)         | Clé correcte (source)      |
|----------|-----------------------------|----------------------------|
| create() | validated_data['name']      | validated_data['nom']      |
| create() | validated_data['price']     | validated_data['prix']     |
| update() | validated_data.get('name')  | validated_data.get('nom')  |
| update() | validated_data.get('price') | validated_data.get('prix') |

#### ArticleIngredientSerializer

| Méthode  | Clé utilisée (faux)                     | Clé correcte (source)                     |
|----------|-----------------------------------------|-------------------------------------------|
| create() | validated_data['required_quantity']     | validated_data['quantite_necessaire']     |
| update() | validated_data.get('required_quantity') | validated_data.get('quantite_necessaire') |

#### ArticleDetailSerializer

| Méthode  | Clé utilisée (faux)         | Clé correcte (source)      |
|----------|-----------------------------|----------------------------|
| create() | validated_data['name']      | validated_data['nom']      |
| create() | validated_data['price']     | validated_data['prix']     |
| update() | validated_data.get('name')  | validated_data.get('nom')  |
| update() | validated_data.get('price') | validated_data.get('prix') |

### 2. apps/inventory/serializers.py

### IngredientSerializer

| Méthode  | Clé utilisée (faux)              | Clé correcte (source)               |
|----------|----------------------------------|-------------------------------------|
| create() | validated_data['name']           | validated_data['nom']               |
| create() | validated_data['unit']           | validated_data['unite']             |
| create() | validated_data['unit_price']     | validated_data['prix_unitaire']     |
| update() | validated_data.get('name')       | validated_data.get('nom')           |
| update() | validated_data.get('unit')       | validated_data.get('unite')         |
| update() | validated_data.get('unit_price') | validated_data.get('prix_unitaire') |

### StockSerializer

| Méthode  | Clé utilisée (faux)                     | Clé correcte (source)                   |
|----------|-----------------------------------------|-----------------------------------------|
| create() | validated_data['quantity_in_stock']     | validated_data['quantite_en_stock']     |
| create() | validated_data.get('alert_threshold')   | validated_data.get('seuil_alerte')      |
| update() | validated_data.get('quantity_in_stock') | validated_data.get('quantite_en_stock') |
| update() | validated_data['alert_threshold']       | validated_data['seuil_alerte']          |

## 3. apps/restaurant/serializers.py

### RestaurantSerializer

| Méthode  | Clé utilisée (faux)                | Clé correcte (source)                    |
|----------|------------------------------------|------------------------------------------|
| create() | validated_data.get('name')         | validated_data.get('nom_restaurant')     |
| create() | validated_data.get('address')      | validated_data.get('adresse_restaurant') |
| create() | validated_data.get('postal_code')  | validated_data.get('code_postal')        |
| create() | validated_data.get('city')         | validated_data.get('ville')              |
| create() | validated_data.get('phone_number') | validated_data.get('numero_telephone')   |
| create() | validated_data.get('siret')        | validated_data.get('numero_siret')       |
| create() | validated_data.get('naf_code')     | validated_data.get('code_naf')           |
| create() | validated_data.get('pin')          | validated_data.get('pin_restaurant')     |
| update() | validated_data.get('name')         | validated_data.get('nom_restaurant')     |
| update() | validated_data.get('address')      | validated_data.get('adresse_restaurant') |
| update() | validated_data.get('postal_code')  | validated_data.get('code_postal')        |
| update() | validated_data.get('city')         | validated_data.get('ville')              |
| update() | validated_data.get('phone_number') | validated_data.get('numero_telephone')   |
| update() | validated_data.get('siret')        | validated_data.get('numero_siret')       |

|          |                                |                                      |
|----------|--------------------------------|--------------------------------------|
| update() | validated_data.get('naf_code') | validated_data.get('code_naf')       |
| update() | validated_data.get('pin')      | validated_data.get('pin_restaurant') |

#### 4. apps/salles/serializers.py

##### SalleSerializer

| Méthode    | Clé utilisée (faux)            | Clé correcte (source)           |
|------------|--------------------------------|---------------------------------|
| validate() | data.get('name')               | data.get('nom_salle')           |
| create()   | validated_data.get('name')     | validated_data.get('nom_salle') |
| create()   | validated_data.get('capacity') | validated_data.get('capacite')  |
| create()   | validated_data.get('floor')    | validated_data.get('etage')     |
| update()   | 'name' in validated_data       | 'nom_salle' in validated_data   |
| update()   | validated_data.get('capacity') | validated_data.get('capacite')  |
| update()   | 'floor' in validated_data      | 'etage' in validated_data       |

#### 5. apps/suppliers/serializers.py

##### CommandeFournisseurSerializer

| Méthode    | Clé utilisée (faux)                          | Clé correcte (source)                       |
|------------|----------------------------------------------|---------------------------------------------|
| create()   | validated_data.get('order_date')             | validated_data.get('date_commande')         |
| create()   | validated_data.get('order_number')           | validated_data.get('numero_commande')       |
| create()   | validated_data.get('expected_delivery_date') | validated_data.get('date_livraison_prevue') |
| create()   | validated_data.get('status')                 | validated_data.get('statut')                |
| create()   | validated_data.get('total_amount')           | validated_data.get('montant_total')         |
| validate() | attrs.get('order_number')                    | attrs.get('numero_commande')                |
| update()   | 'order_number' in validated_data             | 'numero_commande' in validated_data         |
| update()   | 'order_date' in validated_data               | 'date_commande' in validated_data           |
| update()   | 'expected_delivery_date' in validated_data   | 'date_livraison_prevue' in validated_data   |
| update()   | 'status' in validated_data                   | 'statut' in validated_data                  |
| update()   | 'total_amount' in validated_data             | 'montant_total' in validated_data           |

#### 6. apps/reservations/serializers.py

##### ReservationSerializer

| Méthode  | Clé utilisée (faux)           | Clé correcte (source)        |
|----------|-------------------------------|------------------------------|
| create() | validated_data['client_name'] | validated_data['nom_client'] |

|          |                                    |                                        |
|----------|------------------------------------|----------------------------------------|
| create() | validated_data['party_size']       | validated_data['nombre_personnes']     |
| create() | validated_data['datetime']         | validated_data['date_heure']           |
| create() | validated_data.get('phone_number') | validated_data.get('telephone')        |
| update() | validated_data.get('client_name')  | validated_data.get('nom_client')       |
| update() | validated_data.get('party_size')   | validated_data.get('nombre_personnes') |
| update() | validated_data.get('datetime')     | validated_data.get('date_heure')       |
| update() | validated_data.get('phone_number') | validated_data.get('telephone')        |

## 7. apps/billing/serializers.py

### FactureSerializer

| Méthode  | Clé utilisée (faux)         | Clé correcte (source)      |
|----------|-----------------------------|----------------------------|
| create() | validated_data['number']    | validated_data['numero']   |
| create() | validated_data.get('state') | validated_data.get('etat') |
| update() | 'number' in validated_data  | 'numero' in validated_data |
| update() | 'state' in validated_data   | 'etat' in validated_data   |

### PaiementSerializer

| Méthode    | Clé utilisée (faux)          | Clé correcte (source)         |
|------------|------------------------------|-------------------------------|
| validate() | attrs.get('amount')          | attrs.get('montant')          |
| create()   | validated_data.get('amount') | validated_data.get('montant') |

## 8. apps/reports/serializers.py

### ReportSerializer

| Méthode  | Clé utilisée (faux)                | Clé correcte (source)               |
|----------|------------------------------------|-------------------------------------|
| create() | validated_data['report_type']      | validated_data['type_report']       |
| create() | validated_data['period_start']     | validated_data['periode_debut']     |
| create() | validated_data['period_end']       | validated_data['periode_fin']       |
| create() | validated_data.get('file')         | validated_data.get('fichier')       |
| update() | validated_data.get('report_type')  | validated_data.get('type_report')   |
| update() | validated_data.get('period_start') | validated_data.get('periode_debut') |
| update() | validated_data.get('period_end')   | validated_data.get('periode_fin')   |
| update() | 'file' in validated_data           | 'fichier' in validated_data         |

## Erreurs confirmées en production

## Erreur 1 : `POST /api/articles/` → 500 Internal Server Error

Requête envoyée par le frontend :

```
{
  "name": "Recette A",
  "categorie_id": 1,
  "price": "10",
  "description": "aaa"
}
```

Ce que DRF fait en interne :

1. Le serializer `ArticleDetailSerializer` reçoit le JSON
2. Le champ `name = CharField(source='nom')` accepte la valeur `"Recette A"`
3. DRF valide et construit `validated_data` en utilisant les clés **source** :

```
validated_data = {
    'nom': 'Recette A',          # ← clé = source, PAS 'name'
    'prix': Decimal('10.00'),    # ← clé = source, PAS 'price'
    'categorie': <CategorieArticle: Entrées>,
    'description': 'aaa',
}
```

4. La méthode `create()` est appelée avec ce `validated_data`

Le crash :

```
# apps/menu/serializers.py, ligne 269
def create(self, validated_data):
    article = Article.objects.create(
        nom=validated_data['name'],    # ← KeyError: 'name' n'existe pas !
        prix=validated_data['price'],  # ← aurait aussi crashé
        ...
    )
```

Traceback complète :

```
File "apps/menu/serializers.py", line 269, in create
    nom=validated_data['name'],
KeyError: 'name'
Raised during: apps.menu.views.ArticleViewSet
```

**Impact** : aucun article ne peut être créé depuis le frontend web ni l'app mobile.

---

## Erreur 2 : `POST /api/planning/shifts/` → 500 Internal Server Error

Requête envoyée par le frontend :

```
{
  "employe_id": 4,
  "restaurant_id": 3,
  "start_date": "2026-05-14T10:00:00",
  "end_date": "2026-05-14T15:00:00",
}
```

```
"shift_type": "MORNING"
}
```

#### Ce que DRF fait en interne :

1. Le champ `start_date = DateTimeField(source='date_debut')` accepte la valeur
2. `validated_data` contient `'date_debut'`, pas `'start_date'`
3. Le `create()` accède `validated_data['start_date']` → `KeyError`

**Statut : CORRIGÉ dans la branche `production`** — le `create()` / `update()` custom a été supprimé. DRF utilise maintenant le mapping `source` automatiquement. C'est la bonne approche.

#### Erreurs silencieuses (pas de crash, mais données jamais sauvegardées)

Certains serializers utilisent `.get()` avec un fallback, ce qui ne crashe pas mais **ignore silencieusement les nouvelles valeurs** :

```
# apps/restaurant/serializers.py – RestaurantSerializer.update()
def update(self, instance, validated_data):
    instance.nom_restaurant = validated_data.get('name', instance.nom_restaurant)
    #                               ^^^^^^
    # 'name' n'existe pas dans validated_data → retourne instance.nom_restaurant (l'ancienne valeur)
    # → Le champ n'est JAMAIS mis à jour, même si le frontend envoie un nouveau nom
```

**Impact :** les PUT/PATCH sur `/api/restaurants/:id/` retournent 200 OK mais **aucun champ n'est modifié** en base. L'utilisateur pense que la modification a fonctionné, mais un rechargement de page montre les anciennes valeurs.

#### Serializers impactés par ce bug silencieux :

- `RestaurantSerializer` (tous les champs : nom, adresse, ville, téléphone, SIRET, code NAF, PIN)
- `SalleSerializer` (nom, capacité, étage)
- `CommandeFournisseurSerializer` (numéro, date, statut, montant)
- `ReservationSerializer.update()` (nom client, taille groupe, date, téléphone)

#### Conséquences par type d'accès

| Type d'accès                               | Exemple                                               | Conséquence                                                                                                               |
|--------------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Accès direct []                            | <code>validated_data['name']</code>                   | <b>KeyError → 500 Internal Server Error</b> (crash immédiat, la requête échoue)                                           |
| <code>.get()</code> avec fallback instance | <code>validated_data.get('name', instance.nom)</code> | <b>Bug silencieux</b> — retourne toujours l'ancienne valeur, le champ n'est jamais mis à jour. Le frontend reçoit 200 OK. |
| <code>.get()</code> sans fallback          | <code>validated_data.get('name')</code>               | Retourne <code>None</code> — peut causer une <b>IntegrityError</b> si le champ est NOT NULL en base                       |

#### La correction

##### Option 1 (recommandée) : supprimer les `create()` / `update()` custom

Quand ces méthodes ne font que mapper les champs vers le modèle, **elles sont inutiles**. DRF sait déjà faire ce mapping automatiquement grâce à `source`. Supprimer ces méthodes élimine le bug et réduit le code.

C'est exactement ce qui a été fait avec succès pour `ShiftSerializer` et `FournisseurSerializer` dans la branche `production`.

### Exemple — avant :

```
class ArticleSerializer(serializers.ModelSerializer):
    name = serializers.CharField(source='nom')
    price = serializers.DecimalField(source='prix', ...)

    def create(self, validated_data):          # ← SUPPRIMER cette méthode
        return Article.objects.create(
            nom=validated_data['name'],      # ← bug
            prix=validated_data['price'],    # ← bug
        )

    def update(self, instance, validated_data): # ← SUPPRIMER cette méthode
        instance.nom = validated_data.get('name', instance.nom) # ← bug silencieux
        instance.prix = validated_data.get('prix', instance.prix) # ← bug silencieux
        instance.save()
        return instance
```

### Exemple — après (les méthodes sont supprimées, DRF gère tout) :

```
class ArticleSerializer(serializers.ModelSerializer):
    name = serializers.CharField(source='nom')
    price = serializers.DecimalField(source='prix', ...)

    # Pas de create() ni update() custom
    # DRF utilise validated_data directement avec les bonnes clés source
```

### Option 2 : corriger les clés manuellement

Si la méthode `create()` / `update()` fait de la logique custom (relations imbriquées, calculs, etc.), remplacer chaque accès par la clé source :

```
def create(self, validated_data):
    return Article.objects.create(
        nom=validated_data['nom'],          # ← clé source, pas 'name'
        prix=validated_data['prix'],        # ← clé source, pas 'price'
    )
```

### Checklist de correction

Pour chaque serializer de la liste "Toujours cassés" :

- ☐ `CategorieArticleSerializer` — `apps/menu/serializers.py`
- ☐ `ArticleSerializer` — `apps/menu/serializers.py`
- ☐ `ArticleDetailSerializer` — `apps/menu/serializers.py`
- ☐ `ArticleIngredientSerializer` — `apps/menu/serializers.py`
- ☐ `IngredientSerializer` — `apps/inventory/serializers.py`
- ☐ `StockSerializer` — `apps/inventory/serializers.py`
- ☐ `RestaurantSerializer` — `apps/restaurant/serializers.py`
- ☐ `SalleSerializer` — `apps/salles/serializers.py`
- ☐ `ReservationSerializer` — `apps/reservations/serializers.py`
- ☐ `CommandeFournisseurSerializer` — `apps/suppliers/serializers.py`
- ☐ `FactureSerializer` — `apps/billing/serializers.py`

- ☐ PaiementSerializer — apps/billing/serializers.py
- ☐ ReportSerializer — apps/reports/serializers.py